

HOMEWORK 1 (124 points + 1) : CAUTION! CONTENTS ARE HOT

DUE: *September 22, 2026 @ 11:59 PM*

**24-HR LATE DUE DATE WITH A 15% PENALTY:
*September 23, 2026 @ 11:59 PM***

The [NCEI/WDS Global Significant Volcanic Eruptions Database](#) is a very comprehensive collection of +600 volcanic eruptions dating from 4360 BC to the present. Due to the nature of this assignment, we will be dealing with relatively newer volcanoes (in which some are still obviously still older than anyone on Earth currently). Each eruption in the database is classified as significant if it meets one or more criteria, such as causing fatalities, incurring **damage on property (+\$1 million)**, reaching a **Volcanic Explosivity Index (VEI)** of **6 or higher**, generating a tsunami, or being linked to a significant earthquake. The VEI is a scale that measures the explosiveness of volcanic eruptions, providing insight into the magnitude and potential consequences of the eruptions. The database includes detailed information on the location, type of volcano, last known eruption, VEI, casualties, property damage, and much more.



We are going to dive straight into these volcanoes (well... their dataset), to swim our way into Pandas proficiency!

You will find the [Pandas Documentation](#) helpful. There are also some helpful links to guide you along the way! Don't get burned

For this assignment, we are including a special bonus question to help everyone get familiar with the proper assignment submission routine. We strongly encourage you to complete Bonus

Question 1, learn the process, and follow this practice for all future assignments.

For this assignment only, you will not lose points if you skip the bonus question. However, starting from Homework 2, submitting both the Jupyter Notebook file and the PDF will be required in order to receive full credit.

DO NOT REMOVE ANY PART OF ANY OF THE QUESTIONS OR YOU LOSE CREDIT

No Hardcoding either

REMEMBER TO SHOW OUTPUT

Part 1: Maintenance (25 POINTS TOTAL)

First, we're going to familiarize ourselves with the process. As in most languages, Python looks best when its modules are imported first before any other code is written

```
In [2]: # Make sure these code blocks run properly and that you have properly installed
import pandas as pd
import requests
# import other libraries here

# Don't Remove this
pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)
```

```
/Users/riyatakalkar/Library/Python/3.9/lib/python/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
warnings.warn(
```

As you may have noticed, there's another library aside from Pandas called "[requests](#)."

The requests library allows you to send HTTP requests to a server, retrieve the content, and process it at ease. It's very beginner friendly for those attempting to get into web scraping (super important for collecting and creating datasets). We also recommend looking into [BeautifulSoup](#) (yeah, soup LOL), another wonderful library that can be paired with the requests library for web scraping.

As shown below, sometimes specific websites require specific headers in order to process a request to access the data.

To check if a request was processed successfully, use the `status_code` function to see if the process returned 200.

```
In [3]: # API URL and headers in case request gets denied.  
api_url = "https://www.ngdc.noaa.gov/hazel/hazard-service/api/v1/volcanoes"  
  
headers = {  
    'accept': '*/*'   
}
```

TASK 1.0: Cute Webscraping (5 points)

To make our cute webscraper we need to **create a GET request** using the relevant information given above.

This particular dataset NOAA returns data from the API as **json** when a user makes a request. The json data has a particular format, so we will extract our needed information only from the field called **items** to make a dataframe.

After properly scraping the data, name this dataframe *df*

Save this dataframe into a CSV file named 'volcanoes.csv'

You won't need to run this more than once

```
In [5]: response = requests.get(api_url, headers=headers)  
data = response.json()  
df = pd.DataFrame(data['items'])  
df.to_csv('volcanoes.csv', index=False)
```

TASK 1.1: 1 Liner Thingz (3 points)

We need to get an idea of what this dataset is going to look. In order to do that, let's take a look at some of the most **basic things** our dataframe has.

Read the directions carefully and code your answer with only one line of code.

CAN'T USE LOOPS. DO NOT DISPLAY THE DATAFRAME, JUST YOUR CODE OUTPUT HERE.

1.1.1: In one line of code and **using only one single method call**, output the **summary statistics** (count, mean, std, min, max, quartiles) of all numerical features in the dataframe.

Hint: Look for a method that gives a descriptive overview of numerical columns.

In [6]: `df.describe()`

Out [6]:

	id	year	month	day	tsunamiEventId	earthquake
count	200.000000	200.000000	177.000000	155.000000	37.000000	17.
mean	375.635000	1809.800000	6.293785	15.825806	1818.648649	3632
std	306.114755	248.034282	3.469611	9.185073	1742.563313	3015
min	1.000000	-141.000000	1.000000	1.000000	61.000000	421.
25%	78.500000	1774.500000	3.000000	8.000000	613.000000	1328.
50%	348.000000	1880.500000	7.000000	15.000000	1181.000000	2179.
75%	664.250000	1950.000000	9.000000	24.000000	1977.000000	6195.
max	994.000000	1996.000000	12.000000	31.000000	5863.000000	10519.

1.1.2: In one line of code and **using only one single method call**, print the **summary information** of the dataframe, including index dtype, column names, non-null counts, and memory usage.

In [7]: `df.info()`

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 43 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   id                                           200 non-null    int64
1   year                                         200 non-null    int64
2   month                                       177 non-null    float64
3   day                                         155 non-null    float64
4   tsunamiEventId                             37 non-null     float64
5   earthquakeEventId                           17 non-null     float64
6   volcanoLocationId                           200 non-null    int64
7   volcanoLocationNewNum                       200 non-null    int64
8   volcanoLocationNum                         200 non-null    object
9   name                                         200 non-null    object
10  location                                    200 non-null    object
11  country                                    200 non-null    object
12  latitude                                    200 non-null    float64
13  longitude                                   200 non-null    float64
14  elevation                                   200 non-null    int64
15  morphology                                  200 non-null    object
16  agent                                       128 non-null    object
17  deathsTotal                               128 non-null    float64
18  deathsAmountOrderTotal                    180 non-null    float64
19  damageAmountOrderTotal                     59 non-null     float64
20  significant                                200 non-null    bool
21  publish                                    200 non-null    bool
22  eruption                                    200 non-null    bool
23  status                                     200 non-null    object
24  timeErupt                                  200 non-null    object
25  vei                                         192 non-null    float64
26  deathsAmountOrder                         169 non-null    float64
27  damageAmountOrder                         51 non-null     float64
28  housesDestroyedAmountOrderTotal            52 non-null     float64
29  deaths                                     118 non-null    float64
30  injuries                                   30 non-null     float64
31  injuriesAmountOrder                       39 non-null     float64
32  injuriesTotal                             33 non-null     float64
33  injuriesAmountOrderTotal                   41 non-null     float64
34  housesDestroyedAmountOrder                 42 non-null     float64
35  housesDestroyed                           13 non-null     float64
36  housesDestroyedTotal                       16 non-null     float64
37  missingAmountOrder                         3 non-null      float64
38  missingAmountOrderTotal                    3 non-null      float64
39  missing                                    1 non-null      float64
40  missingTotal                              1 non-null      float64
41  damageMillionsDollars                      1 non-null      float64
42  damageMillionsDollarsTotal                 1 non-null      float64
dtypes: bool(3), float64(27), int64(5), object(8)
memory usage: 63.2+ KB

```

We won't be using some of the data because there is a lot of missing data.

1.1.3: In one line of code, create a **new dataframe** called **new_df** that **discards** all the features of the **old** dataframe **except for the following**:

```
id, year, month, day, tsunamiEventId, earthquakeEventId,
volcanoLocationId, volcanoLocationNewNum, name, country, elevation,
morphology, deathsTotal, vei, deaths
```

Hint: Don't use any drop function

```
In [9]: new_df = df[['id', 'year', 'month', 'day', 'tsunamiEventId', 'earthquakeEventId', 'volcanoLocationId', 'volcanoLocationNewNum', 'name', 'country', 'elevation', 'morphology', 'deathsTotal', 'vei', 'deaths']]
```

TASK 1.2: 1 Liner Shenaniganz (7 points)

We're going to tidy up the **new dataframe** a little more with some more advanced 1 liner code.

Read the directions carefully and code your answer with only one line of code.

For this section, keep the method of display that is already in the box. Write your code as indicated.

YOU CAN'T USE ONE LINE LOOPS OR ANY KIND OF LOOP.

1.2.1: *In one line of code, **drop any row** that contains **NaN** in **any one** of the columns indicating a measure of **time**.*

```
In [16]: new_df = df.dropna(subset=['year', 'month', 'day']).copy()
```

1.2.2: *In one line of code, **change** the **index column** of the dataframe so that it has **1-based indexing**.*

```
In [17]: new_df.index = range(1, len(new_df) + 1)
```

The **deathsTotal** and **deaths** columns have approximations of the same data with alternating NaNs in each.

1.2.3: *In one line of code, make a **new column** called **'totalDeaths'** that takes the **max** of the values given between those **two** *columns.*

- If there is **NaN** in **one column** and a **numerical** value in the **other**, it will **take the numerical value**.
- **Only** if there are **NaNs** in **both columns**, the **new column** will have **NaN**.

```
In [18]: new_df['totalDeaths'] = new_df[['deathsTotal', 'deaths']].max(axis=1)
```

TASK 1.3: Tailoring Time (10 Points)

It's pretty obvious that the year, month, and day look pretty weird in the dataset. We're going to have to do some hardcore cleaning on the [time](#).

We need to have only ONE column called "date" that contains the full date (YYYY-MM-DD), not separated into three columns.

Make sure there are no floating point values in the date and sort the data from most recent to least.

Remove the old columns and place the new column next to the 'id' column.

YOU CAN USE MULTIPLE LINES OF CODE BUT CAN'T USE LOOPS.

Note: It is alright to have only a **maximum of 12 NaTs** for some dates that often go further back than the 1600s because the datetime module in Pandas has a limit (unless otherwise guided).

```
In [19]: new_df['date'] = pd.to_datetime(new_df[['year', 'month', 'day']], errors='coerce')
# Drop original columns and sort.
new_df.drop(columns=['year', 'month', 'day'], inplace=True)
new_df.sort_values(by='date', ascending=False, inplace=True)
# Place new column next to 'id' column.
date = new_df.pop('date')
new_df.insert(1, 'date', date)

# KEEP THIS. It will display the whole dataframe.
new_df
```

Out[19]:

	id	date	tsunamiEventId	earthquakeEventId	volcanoLocationId	volcanoLocat
122	693	1996-08-10	NaN	NaN	70202	
51	112	1994-11-22	NaN	NaN	60325	
155	933	1991-06-03	NaN	NaN	80210	
61	144	1990-02-10	NaN	NaN	60328	
93	486	1988-05-09	NaN	NaN	60509	
25	31	1987-04-17	NaN	NaN	10106	
50	111	1986-12-31	NaN	NaN	60325	
9	13	1986-07-24	NaN	NaN	10104	
72	203	1985-05-10	NaN	NaN	60330	
135	739	1984-09-09	NaN	NaN	70303	
34	40	1984-08-15	5807.0	NaN	20403	
88	447	1983-08-17	4702.0	NaN	60425	
94	488	1983-07-23	NaN	NaN	60601	
84	398	1981-09-05	NaN	NaN	60415	
75	206	1981-03-29	NaN	NaN	60330	
24	30	1979-09-12	NaN	NaN	10106	
15	20	1979-03-08	NaN	NaN	50103	

	id	date	tsunamiEventId	earthquakeEventId	volcanoLocationId	volcanoLocat
39	49	1979-02-20	NaN	NaN	60320	
73	204	1978-09-19	NaN	NaN	60330	
124	698	1978-07-29	NaN	NaN	70301	
5	7	1977-01-10	NaN	NaN	20303	
100	562	1976-09-15	NaN	NaN	60702	
74	205	1976-08-31	NaN	NaN	60330	
99	561	1974-02-11	NaN	NaN	60702	
16	21	1970-03-02	NaN	NaN	10101	
142	777	1969-03-21	2872.0	NaN	70402	
81	389	1969-01-28	NaN	NaN	60411	
49	105	1969-01-07	NaN	NaN	60325	
134	736	1968-04-21	NaN	NaN	70303	
71	202	1967-08-31	NaN	NaN	60330	
107	589	1966-08-12	NaN	NaN	60704	
60	141	1966-04-26	NaN	NaN	60328	
141	762	1965-09-28	1977.0	NaN	70307	
83	395	1964-01-01	NaN	NaN	60415	
77	345	1963-09-05	NaN	NaN	60401	

	id	date	tsunamiEventId	earthquakeEventId	volcanoLocationId	volcanoLocat
79	353	1963-05-16	NaN	4292.0	60402	
70	201	1963-05-05	NaN	NaN	60330	
78	351	1963-03-18	1942.0	NaN	60402	
115	657	1962-12-31	NaN	NaN	60806	
48	104	1961-05-08	NaN	NaN	60325	
96	525	1958-07-12	NaN	NaN	60611	
150	878	1955-10-13	NaN	NaN	80208	
32	38	1954-08-03	NaN	NaN	50101	
47	100	1954-01-18	NaN	NaN	60325	
59	140	1951-08-31	NaN	NaN	60328	
121	680	1950-09-15	NaN	NaN	70108	
69	200	1950-08-28	NaN	NaN	60330	
87	441	1948-04-07	NaN	NaN	60425	
133	735	1947-01-07	NaN	NaN	70303	
68	199	1946-10-29	NaN	NaN	60330	
38	45	1944-12-04	NaN	NaN	60320	
33	39	1944-03-27	NaN	NaN	10102	
67	196	1941-09-21	NaN	NaN	60330	

	id	date	tsunamiEventId	earthquakeEventId	volcanoLocationId	volcanoLocat
98	552	1940-06-20	NaN	NaN	60702	
132	733	1938-06-05	NaN	NaN	70303	
123	697	1933-12-25	1666.0	NaN	70301	
144	823	1933-12-24	NaN	NaN	80205	
46	94	1930-12-18	NaN	NaN	60325	
20	26	1930-09-11	4218.0	NaN	10104	
22	28	1928-11-08	NaN	NaN	10106	
82	394	1928-08-04	1607.0	NaN	60415	
35	42	1926-01-23	NaN	NaN	10204	
45	92	1920-07-25	NaN	NaN	60325	
19	25	1919-05-22	1476.0	NaN	10104	
58	138	1919-05-19	NaN	NaN	60328	
14	18	1917-04-01	NaN	NaN	40105	
26	32	1914-09-10	NaN	NaN	40104	
149	867	1914-01-12	1412.0	2994.0	80208	
66	195	1913-06-23	NaN	NaN	60330	
106	585	1913-03-14	1407.0	2973.0	60704	
28	34	1912-12-03	NaN	NaN	20302	

	id	date	tsunamiEventId	earthquakeEventId	volcanoLocationId	volcanoLocat
65	193	1911-11-08	NaN	NaN	60330	
140	761	1911-01-30	1392.0	8391.0	70307	
64	191	1909-08-29	NaN	NaN	60330	
86	411	1907-09-28	NaN	NaN	60418	
7	11	1907-08-04	NaN	NaN	201112	
8	12	1905-03-10	NaN	NaN	10102	
11	15	1904-02-25	NaN	NaN	30301	
21	27	1903-08-30	NaN	NaN	40105	
57	137	1901-05-22	NaN	NaN	60328	
131	730	1897-05-23	NaN	NaN	70303	
29	35	1895-06-17	NaN	NaN	50103	
63	176	1895-05-22	NaN	NaN	60330	
105	584	1892-06-07	1197.0	NaN	60704	
118	669	1890-06-29	NaN	NaN	60807	
101	569	1889-09-06	1181.0	6309.0	60703	
3	4	1888-03-13	1175.0	NaN	50107	

	id	date	tsunamiEventId	earthquakeEventId	volcanoLocationId	volcanoLocat
130	724	1887-03-09	NaN	NaN	70303	
27	33	1886-08-31	NaN	NaN	40311	
62	167	1885-04-18	NaN	NaN	60330	
12	16	1878-02-04	1095.0	6195.0	50214	
56	136	1875-01-29	NaN	NaN	60328	
139	756	1874-07-19	NaN	NaN	70307	
119	677	1873-01-16	NaN	NaN	70106	
44	82	1872-11-03	NaN	NaN	60325	
23	29	1872-04-24	NaN	NaN	10102	
43	81	1872-04-15	NaN	NaN	60325	
129	716	1871-12-08	NaN	NaN	70303	
114	643	1871-08-07	NaN	NaN	60806	
120	679	1871-04-30	1020.0	2179.0	70108	
97	533	1871-03-03	1022.0	NaN	60701	
85	410	1869-07-07	NaN	NaN	60418	
55	135	1864-01-04	NaN	NaN	60328	
117	666	1861-12-29	NaN	NaN	60807	
104	581	1856-03-02	859.0	1995.0	60704	

	id	date	tsunamiEventId	earthquakeEventId	volcanoLocationId	volcanoLocat
143	787	1853-10-29	809.0	NaN	80103	
128	712	1853-07-13	NaN	NaN	70303	
31	37	1853-06-24	NaN	NaN	40311	
54	133	1848-05-16	NaN	NaN	60328	
95	489	1845-02-08	741.0	1865.0	60603	
37	44	1843-11-25	NaN	NaN	10106	
113	629	1840-02-02	718.0	NaN	60806	
112	627	1838-02-26	NaN	NaN	60806	
42	71	1832-12-25	NaN	NaN	60325	
4	6	1832-11-01	NaN	NaN	10106	
53	132	1826-10-13	NaN	NaN	60328	
41	69	1822-12-27	NaN	NaN	60325	
92	482	1820-06-11	NaN	NaN	60509	
76	333	1817-01-24	NaN	NaN	60335	
80	367	1815-04-10	613.0	NaN	60404	
127	707	1814-02-01	NaN	NaN	70303	
103	580	1812-08-06	NaN	NaN	60704	
18	24	1805-08-11	NaN	NaN	10102	

	id	date	tsunamiEventId	earthquakeEventId	volcanoLocationId	volcanoLocat
126	706	1800-10-30	NaN	NaN	70303	
154	932	1792-05-21	568.0	1515.0	80210	
148	859	1781-04-11	529.0	NaN	80208	
147	858	1779-11-08	NaN	NaN	80208	
6	9	1779-08-08	NaN	NaN	10102	
111	618	1773-02-02	NaN	NaN	60806	
110	617	1772-05-09	504.0	NaN	60806	
125	704	1766-07-20	NaN	NaN	70303	
138	754	1754-11-28	2853.0	1328.0	70307	
137	753	1749-08-11	431.0	1304.0	70307	
13	17	1737-05-20	NaN	NaN	10102	
153	916	1717-02-07	NaN	NaN	80209	
152	915	1716-11-09	NaN	NaN	80209	
136	752	1716-09-24	387.0	1205.0	70307	
52	128	1716-07-20	NaN	NaN	60328	
102	578	1711-12-11	NaN	NaN	60704	
91	477	1694-11-20	NaN	NaN	60509	

	id	date	tsunamiEventId	earthquakeEventId	volcanoLocationId	volcanoLocat
90	466	1692-06-04	NaN	NaN	60507	
17	23	1682-08-12	NaN	NaN	10102	
1	1	NaT	2852.0	421.0	10106	
2	2	NaT	NaN	NaN	10106	
10	14	NaT	NaN	NaN	10106	
30	36	NaT	NaN	NaN	10101	
36	43	NaT	NaN	920.0	201141	
40	64	NaT	NaN	NaN	60325	
89	457	NaT	285.0	NaN	60505	
108	599	NaT	312.0	NaN	60804	
109	607	NaT	222.0	NaN	60806	
116	664	NaT	5754.0	10519.0	60807	
145	844	NaT	61.0	7341.0	80208	
146	846	NaT	NaN	NaN	80208	
151	904	NaT	NaN	NaN	80209	

Part 2: Volcanic Matryoshkas (30 POINTS TOTAL)

Now, that most of the data has been tidied up. We will organize the data into more sizable pieces of information in order to extract useful information.

2.1.1: (10 points here)

Use the **groupby function in Pandas** to create separate dataframes for each unique country.

- **Each table must only have the columns:** 'date' 'country', 'name', and 'vei' (Having the index is ok. Up to you.)

- **Sort** the dataframe of **each country** by **highest to lowest 'vei'**
- Use the **display** function to show **each sorted table**

You MUST use the groupby function here and display your results.

```
In [21]: from IPython.display import display
for country, country_df in new_df.groupby('country'):
    country_df = country_df[['date', 'country', 'name', 'vei']].sort_values(
        display(country_df))
```

	date	country	name	vei
34	1984-08-15	Cameroon	Oku Volcanic Field	NaN

	date	country	name	vei
11	1904-02-25	Comoros	Karthala	2.0

	date	country	name	vei
28	1912-12-03	Congo, DRC	Nyamulagira	3.0
5	1977-01-10	Congo, DRC	Nyiragongo	1.0

	date	country	name	vei
7	1907-08-04	Ethiopia	Alayta	2.0
36	NaT	Ethiopia	Dama Ali	NaN

	date	country	name	vei
35	1926-01-23	Greece	Santorini	2.0

	date	country	name	vei
80	1815-04-10	Indonesia	Tambora	7.0
108	NaT	Indonesia	Gamkonora	5.0
78	1963-03-18	Indonesia	Agung	5.0
58	1919-05-19	Indonesia	Kelud	4.0
107	1966-08-12	Indonesia	Awu	4.0
53	1826-10-13	Indonesia	Kelud	4.0
117	1861-12-29	Indonesia	Kie Besi	4.0
43	1872-04-15	Indonesia	Merapi	4.0
44	1872-11-03	Indonesia	Merapi	4.0
61	1990-02-10	Indonesia	Kelud	4.0
59	1951-08-31	Indonesia	Kelud	4.0
60	1966-04-26	Indonesia	Kelud	4.0
116	NaT	Indonesia	Kie Besi	4.0
94	1983-07-23	Indonesia	Colo	4.0
81	1969-01-28	Indonesia	Iya	3.0
46	1930-12-18	Indonesia	Merapi	3.0
65	1911-11-08	Indonesia	Semeru	3.0
57	1901-05-22	Indonesia	Kelud	3.0
105	1892-06-07	Indonesia	Awu	3.0
75	1981-03-29	Indonesia	Semeru	3.0
104	1856-03-02	Indonesia	Awu	3.0
54	1848-05-16	Indonesia	Kelud	3.0
113	1840-02-02	Indonesia	Gamalama	3.0
41	1822-12-27	Indonesia	Merapi	3.0
72	1985-05-10	Indonesia	Semeru	3.0
103	1812-08-06	Indonesia	Awu	3.0
110	1772-05-09	Indonesia	Gamalama	3.0
102	1711-12-11	Indonesia	Awu	3.0
40	NaT	Indonesia	Merapi	3.0
109	NaT	Indonesia	Gamalama	3.0
82	1928-08-04	Indonesia	Paluweh	3.0
51	1994-11-22	Indonesia	Merapi	3.0

	date	country	name	vei
79	1963-05-16	Indonesia	Agung	3.0
71	1967-08-31	Indonesia	Semeru	3.0
48	1961-05-08	Indonesia	Merapi	3.0
99	1974-02-11	Indonesia	Karangetang	3.0
73	1978-09-19	Indonesia	Semeru	3.0
70	1963-05-05	Indonesia	Semeru	2.0
115	1962-12-31	Indonesia	Gamalama	2.0
95	1845-02-08	Indonesia	Soputan	2.0
100	1976-09-15	Indonesia	Karangetang	2.0
112	1838-02-26	Indonesia	Gamalama	2.0
42	1832-12-25	Indonesia	Merapi	2.0
67	1941-09-21	Indonesia	Semeru	2.0
76	1817-01-24	Indonesia	Ijen	2.0
55	1864-01-04	Indonesia	Kelud	2.0
77	1963-09-05	Indonesia	Batur	2.0
111	1773-02-02	Indonesia	Gamalama	2.0
83	1964-01-01	Indonesia	Paluweh	2.0
52	1716-07-20	Indonesia	Kelud	2.0
74	1976-08-31	Indonesia	Semeru	2.0
50	1986-12-31	Indonesia	Merapi	2.0
96	1958-07-12	Indonesia	Mahawu	2.0
85	1869-07-07	Indonesia	Lewotobi	2.0
98	1940-06-20	Indonesia	Karangetang	2.0
97	1871-03-03	Indonesia	Ruang	2.0
38	1944-12-04	Indonesia	Dieng Volcanic Complex	2.0
68	1946-10-29	Indonesia	Semeru	2.0
45	1920-07-25	Indonesia	Merapi	2.0
66	1913-06-23	Indonesia	Semeru	2.0
106	1913-03-14	Indonesia	Awu	2.0
87	1948-04-07	Indonesia	Iliwerung	2.0
64	1909-08-29	Indonesia	Semeru	2.0
86	1907-09-28	Indonesia	Lewotobi	2.0

	date	country	name	vei
63	1895-05-22	Indonesia	Semeru	2.0
49	1969-01-07	Indonesia	Merapi	2.0
118	1890-06-29	Indonesia	Kie Besi	2.0
101	1889-09-06	Indonesia	Banua Wuhu	2.0
62	1885-04-18	Indonesia	Semeru	2.0
47	1954-01-18	Indonesia	Merapi	2.0
84	1981-09-05	Indonesia	Paluweh	2.0
114	1871-08-07	Indonesia	Gamalama	2.0
88	1983-08-17	Indonesia	Iliwerung	1.0
39	1979-02-20	Indonesia	Dieng Volcanic Complex	1.0
69	1950-08-28	Indonesia	Semeru	1.0
56	1875-01-29	Indonesia	Kelud	NaN

	date	country	name	vei
30	NaT	Italy	Campi Flegrei	3.0
6	1779-08-08	Italy	Vesuvius	3.0
24	1979-09-12	Italy	Etna	3.0
33	1944-03-27	Italy	Vesuvius	3.0
20	1930-09-11	Italy	Stromboli	3.0
10	NaT	Italy	Etna	3.0
19	1919-05-22	Italy	Stromboli	3.0
2	NaT	Italy	Etna	3.0
23	1872-04-24	Italy	Vesuvius	3.0
17	1682-08-12	Italy	Vesuvius	3.0
13	1737-05-20	Italy	Vesuvius	3.0
25	1987-04-17	Italy	Etna	2.0
18	1805-08-11	Italy	Vesuvius	2.0
4	1832-11-01	Italy	Etna	2.0
9	1986-07-24	Italy	Stromboli	2.0
8	1905-03-10	Italy	Vesuvius	2.0
37	1843-11-25	Italy	Etna	2.0
22	1928-11-08	Italy	Etna	1.0
16	1970-03-02	Italy	Campi Flegrei	NaN
1	NaT	Italy	Etna	NaN

	date	country	name	vei
146	NaT	Japan	Aira	5.0
144	1933-12-24	Japan	Kuchinoerabujima	4.0
149	1914-01-12	Japan	Aira	4.0
148	1781-04-11	Japan	Aira	4.0
147	1779-11-08	Japan	Aira	4.0
150	1955-10-13	Japan	Aira	3.0
153	1717-02-07	Japan	Kirishimayama	3.0
152	1716-11-09	Japan	Kirishimayama	3.0
145	NaT	Japan	Aira	3.0
151	NaT	Japan	Kirishimayama	3.0
154	1792-05-21	Japan	Unzendake	2.0
155	1991-06-03	Japan	Unzendake	1.0

	date	country	name	vei
14	1917-04-01	New Zealand	Okataina	1.0
21	1903-08-30	New Zealand	Okataina	1.0
26	1914-09-10	New Zealand	Whakaari/White Island	NaN

	date	country	name	vei
90	1692-06-04	Pacific Ocean	Serua	4.0
89	NaT	Pacific Ocean	Teon	4.0
93	1988-05-09	Pacific Ocean	Banda Api	3.0
91	1694-11-20	Pacific Ocean	Banda Api	3.0
92	1820-06-11	Pacific Ocean	Banda Api	2.0

	date	country	name	vei
12	1878-02-04	Papua New Guinea	Rabaul	3.0
15	1979-03-08	Papua New Guinea	Karkar	2.0
32	1954-08-03	Papua New Guinea	Bam	2.0
29	1895-06-17	Papua New Guinea	Karkar	2.0
3	1888-03-13	Papua New Guinea	Ritter Island	2.0

	date	country	name	vei
136	1716-09-24	Philippines	Taal	4.0
138	1754-11-28	Philippines	Taal	4.0
141	1965-09-28	Philippines	Taal	4.0
127	1814-02-01	Philippines	Mayon	4.0
131	1897-05-23	Philippines	Mayon	3.0
137	1749-08-11	Philippines	Taal	3.0
125	1766-07-20	Philippines	Mayon	3.0
128	1853-07-13	Philippines	Mayon	3.0
129	1871-12-08	Philippines	Mayon	3.0
135	1984-09-09	Philippines	Mayon	3.0
130	1887-03-09	Philippines	Mayon	3.0
140	1911-01-30	Philippines	Taal	3.0
121	1950-09-15	Philippines	Camiguin	3.0
134	1968-04-21	Philippines	Mayon	3.0
123	1933-12-25	Philippines	Bulusan	2.0
139	1874-07-19	Philippines	Taal	2.0
119	1873-01-16	Philippines	Ragang	2.0
132	1938-06-05	Philippines	Mayon	2.0
120	1871-04-30	Philippines	Camiguin	2.0
133	1947-01-07	Philippines	Mayon	2.0
126	1800-10-30	Philippines	Mayon	2.0
142	1969-03-21	Philippines	Didicas	2.0
124	1978-07-29	Philippines	Bulusan	2.0
122	1996-08-10	Philippines	Kanlaon	2.0

	date	country	name	vei
143	1853-10-29	Taiwan	Unnamed	2.0

	date	country	name	vei
27	1886-08-31	Tonga	Niuafu'ou	4.0
31	1853-06-24	Tonga	Niuafu'ou	0.0

2.1.2: (5 points here)

Using **groupby** again, print out the **maximum 'vei'** for **each unique country**.

You MUST use the groupby function here and print your results.

- **Print** out your results in a format like the following: "Country: {country_name}, Highest VEI: {vei}"

```
In [22]: for country, group in new_df.groupby('country'):
          print(f"Country: {country}, Highest VEI: {group['vei'].max()}")
```

```
Country: Cameroon, Highest VEI: nan
Country: Comoros, Highest VEI: 2.0
Country: Congo, DRC, Highest VEI: 3.0
Country: Ethiopia, Highest VEI: 2.0
Country: Greece, Highest VEI: 2.0
Country: Indonesia, Highest VEI: 7.0
Country: Italy, Highest VEI: 3.0
Country: Japan, Highest VEI: 5.0
Country: New Zealand, Highest VEI: 1.0
Country: Pacific Ocean, Highest VEI: 4.0
Country: Papua New Guinea, Highest VEI: 3.0
Country: Philippines, Highest VEI: 4.0
Country: Taiwan, Highest VEI: 2.0
Country: Tonga, Highest VEI: 4.0
```

Finally, we have ALMOST REACHED THE END!! Since there is still quite a bit of missing data, we want to make use of what is still available.

A very powerful tool in Python's magnificent collection of libraries is its beautiful graphing tools.

Check out libraries such as [Seaborn](#) or [Matplotlib](#) to create meaningful visualizations!

Your final task in this section requires the use of these libraries

2.1.3: (15 points here)

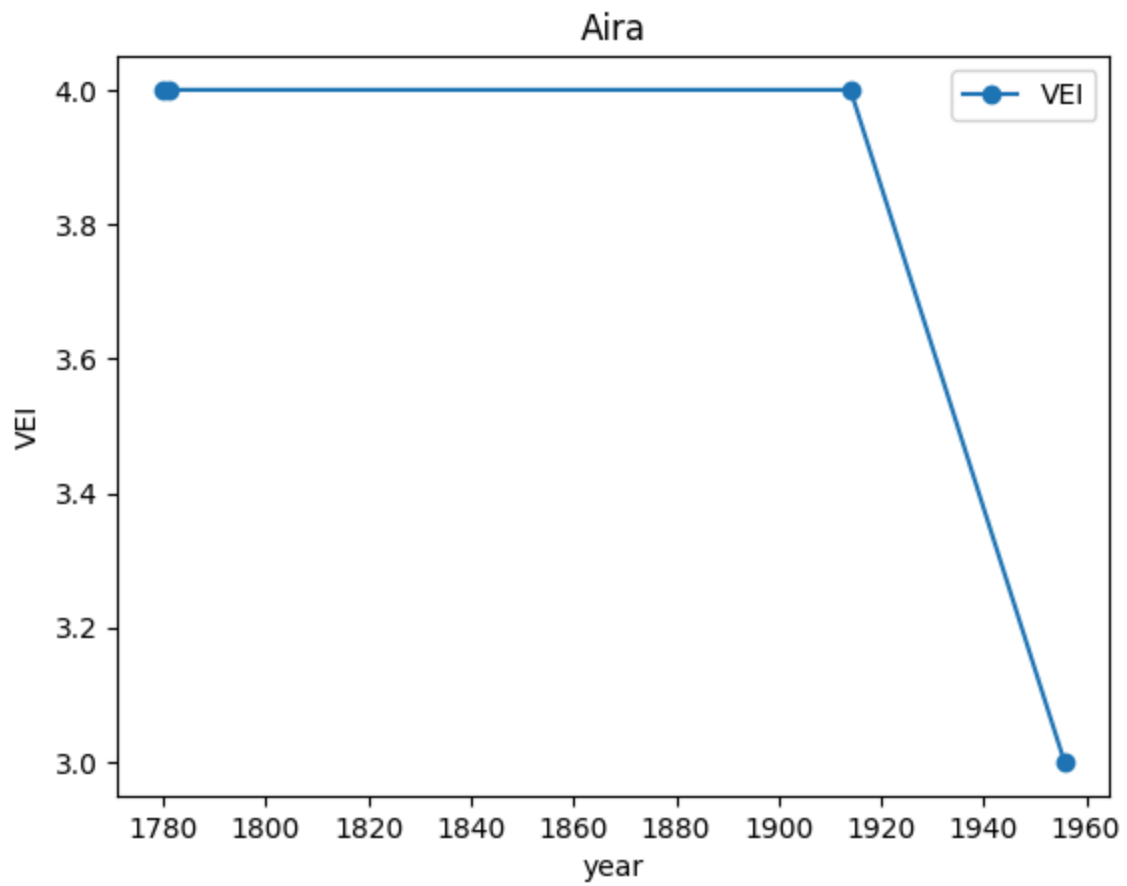
- Based on the **unique names of volcanoes**, **filter names that have more than 4 datapoints under their name**.
- Each datapoint in the dataframe refers to a recorded instance of a volcanic eruption.
- Make **separate line graphs for each volcano** and **plot their VEIs over time**.

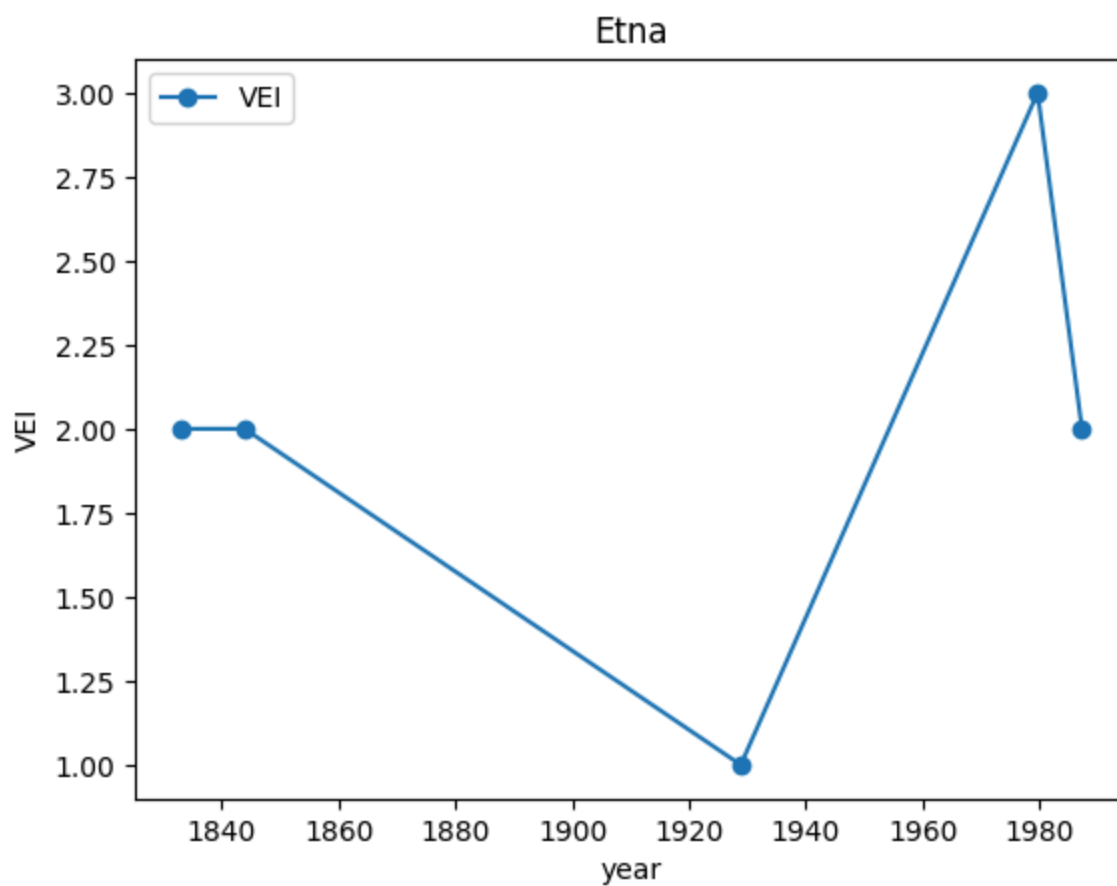
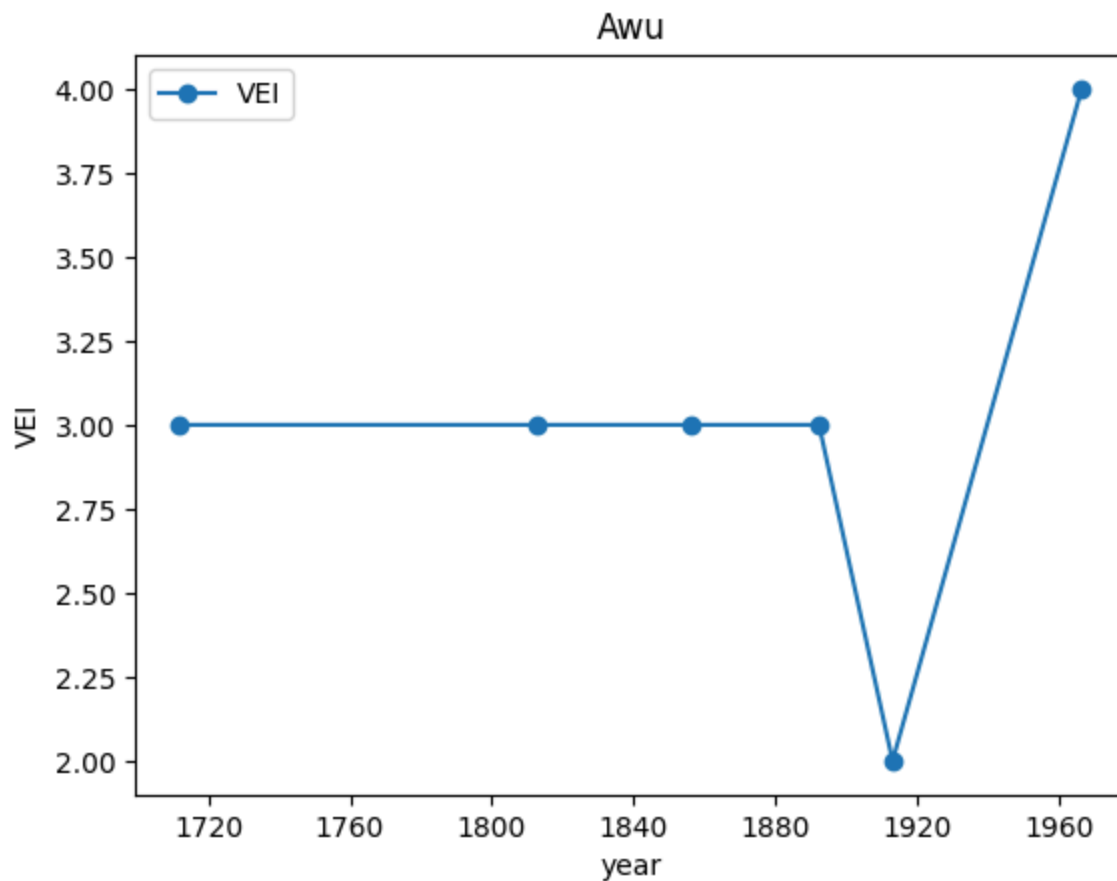
Make sure to properly label all parts of the graph appropriately to receive credit 🙄
(like title, axes, legend, etc...)

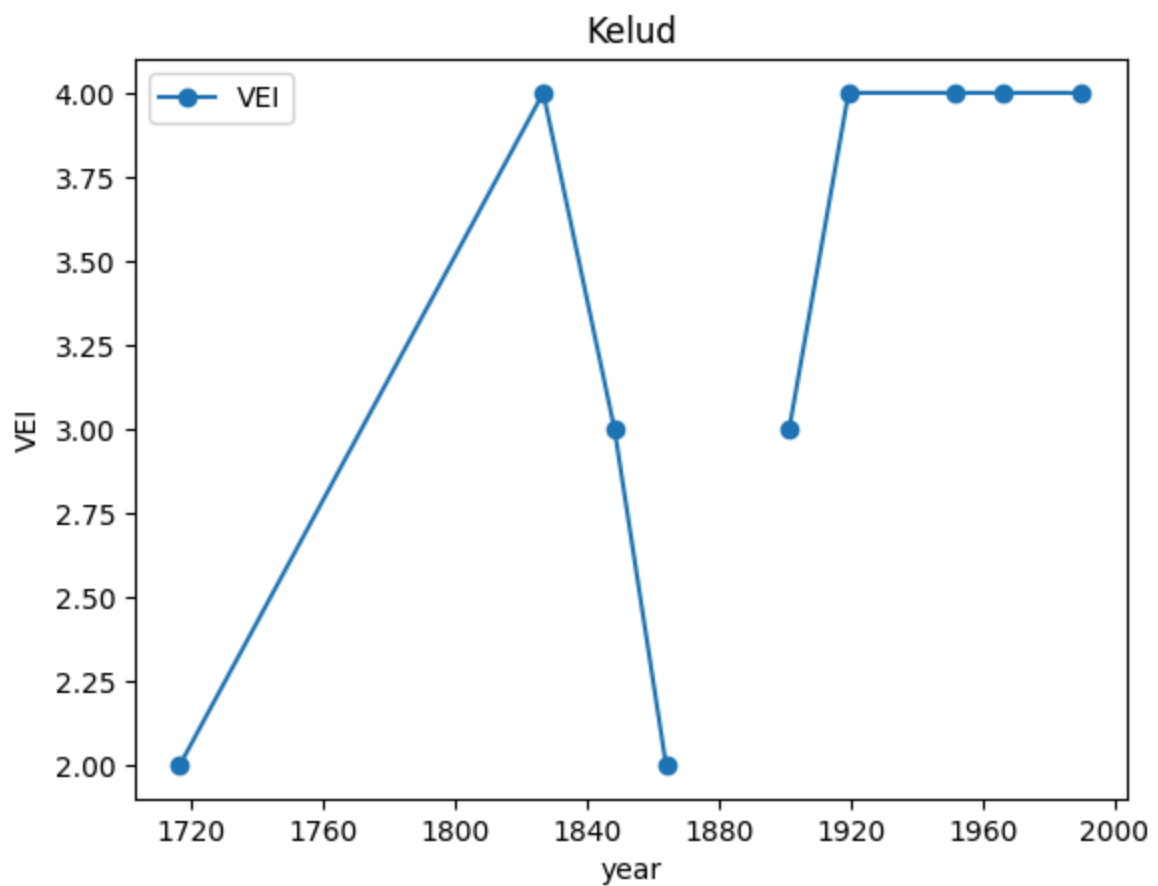
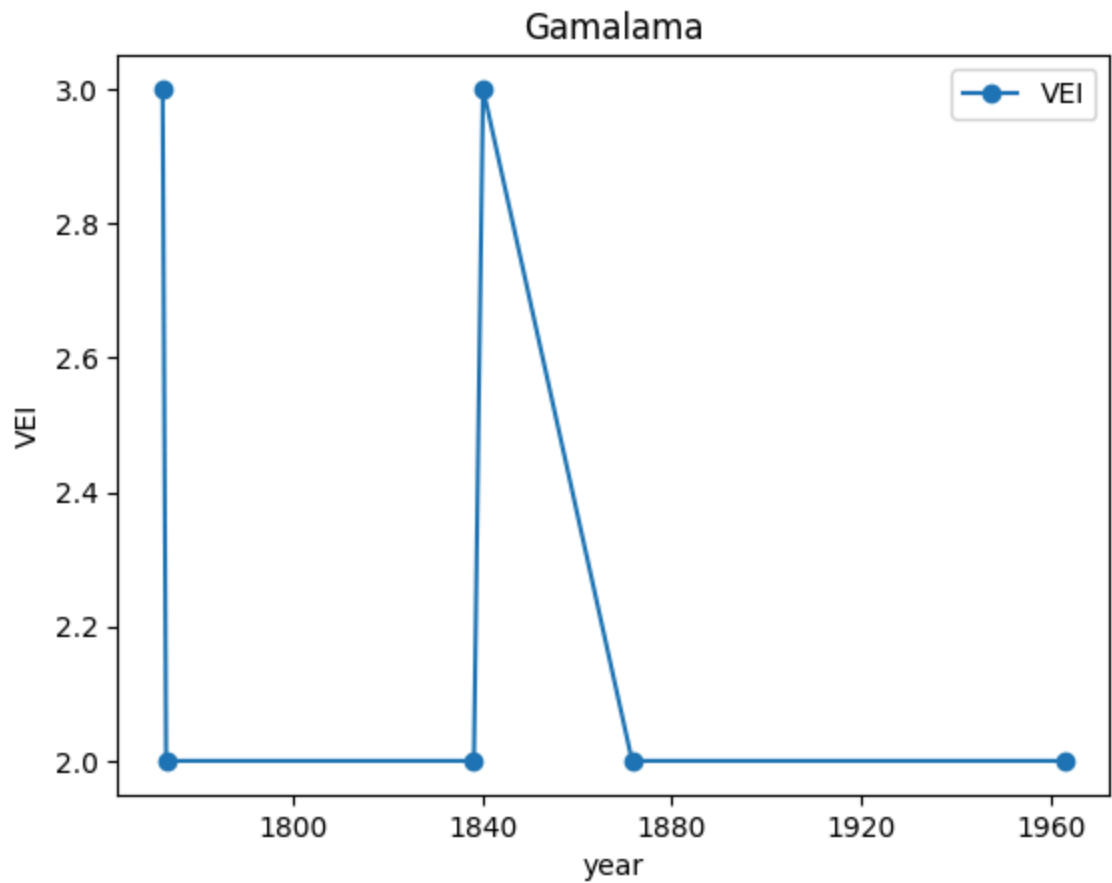
```
In [26]: import matplotlib.pyplot as mpl
          v_counts = new_df.groupby('name').size()
          v_names = v_counts[v_counts > 4].index

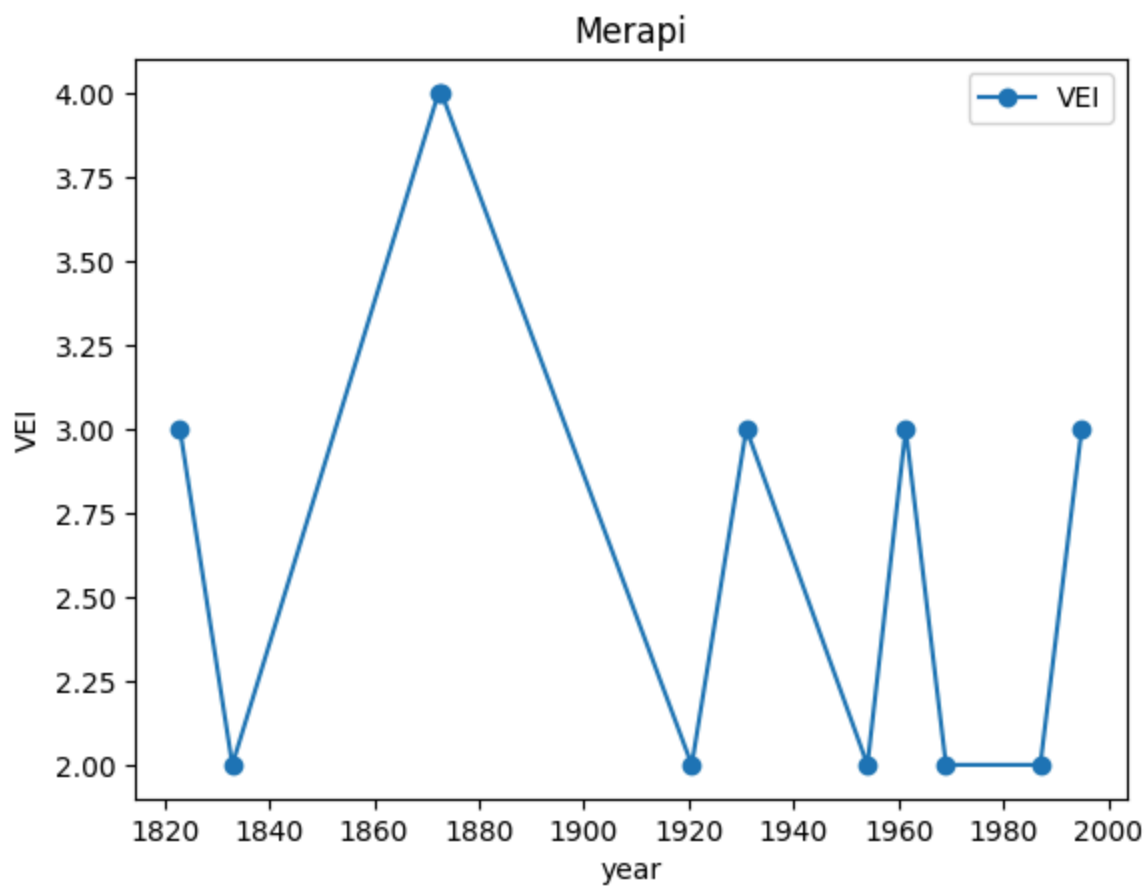
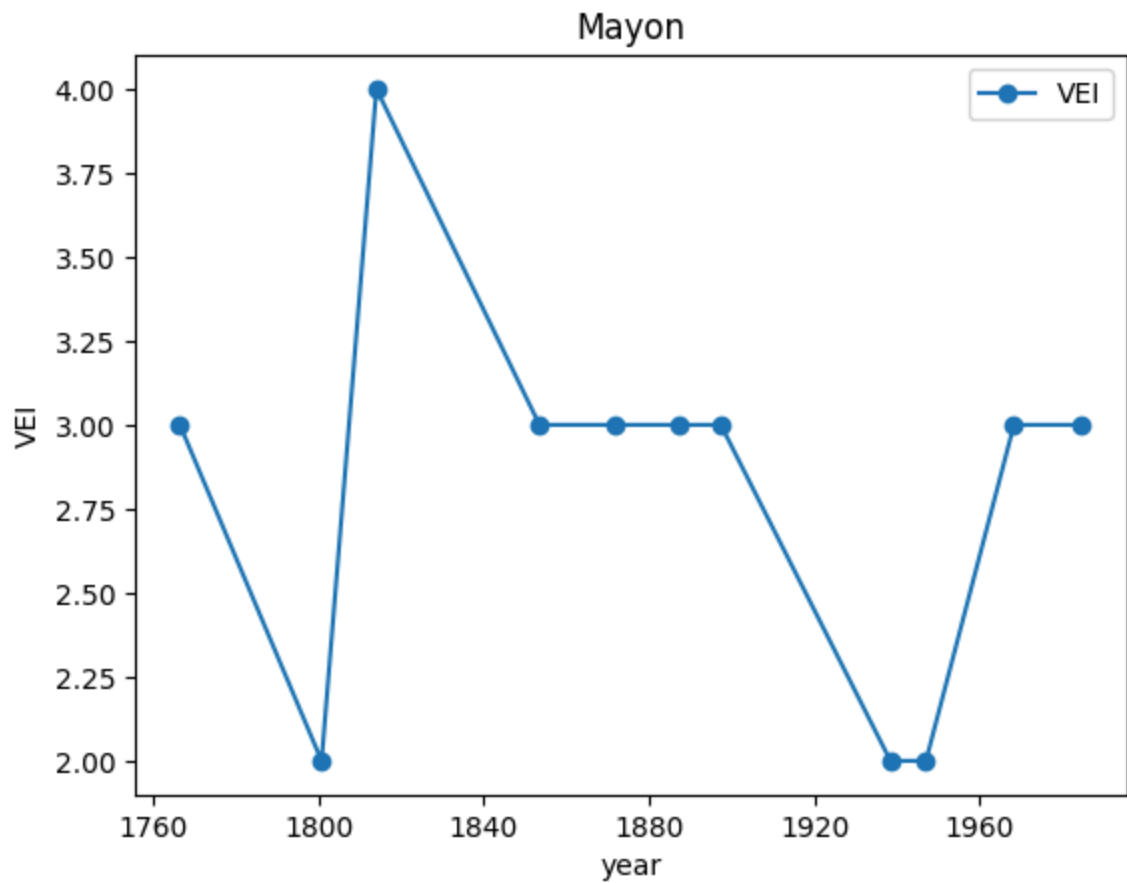
          for volcano in v_names:
              volcano_df = new_df[new_df['name'] == volcano].sort_values('date')
              mpl.figure()
              mpl.plot(volcano_df['date'], volcano_df['vei'], marker='o')
```

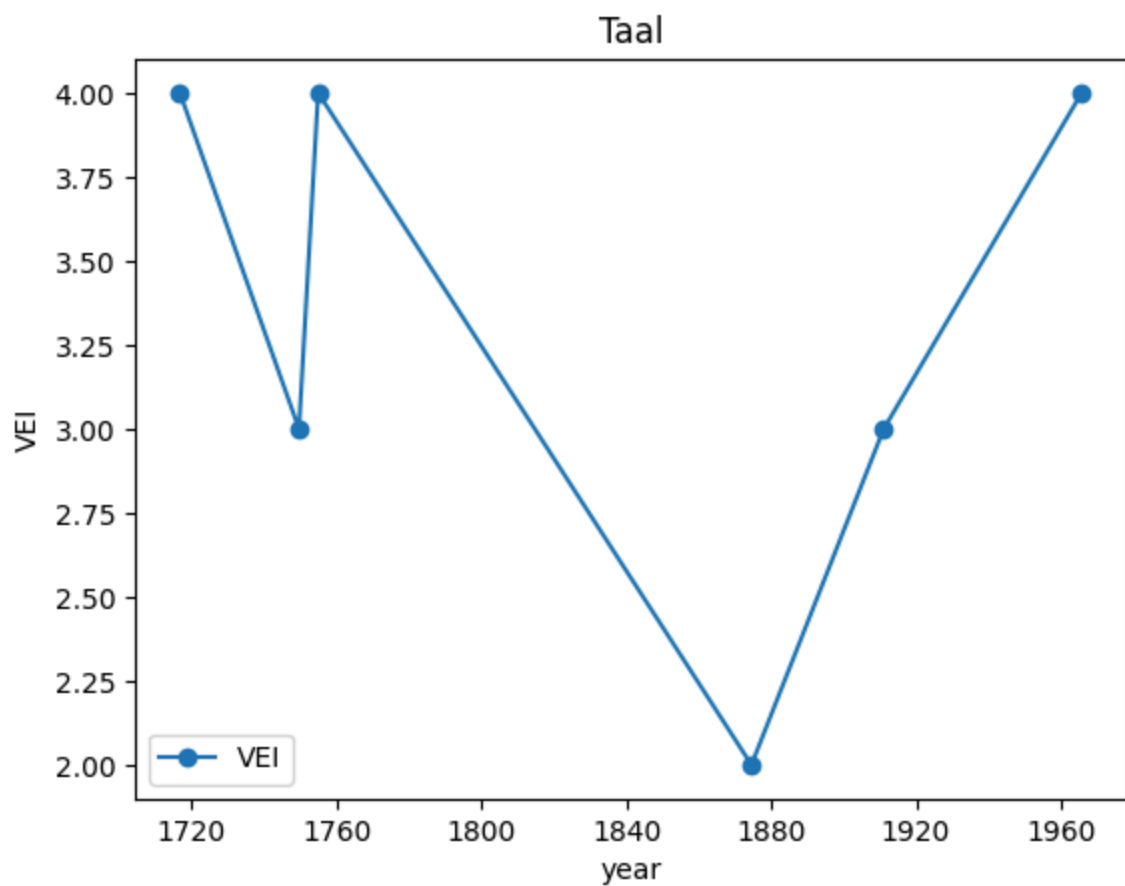
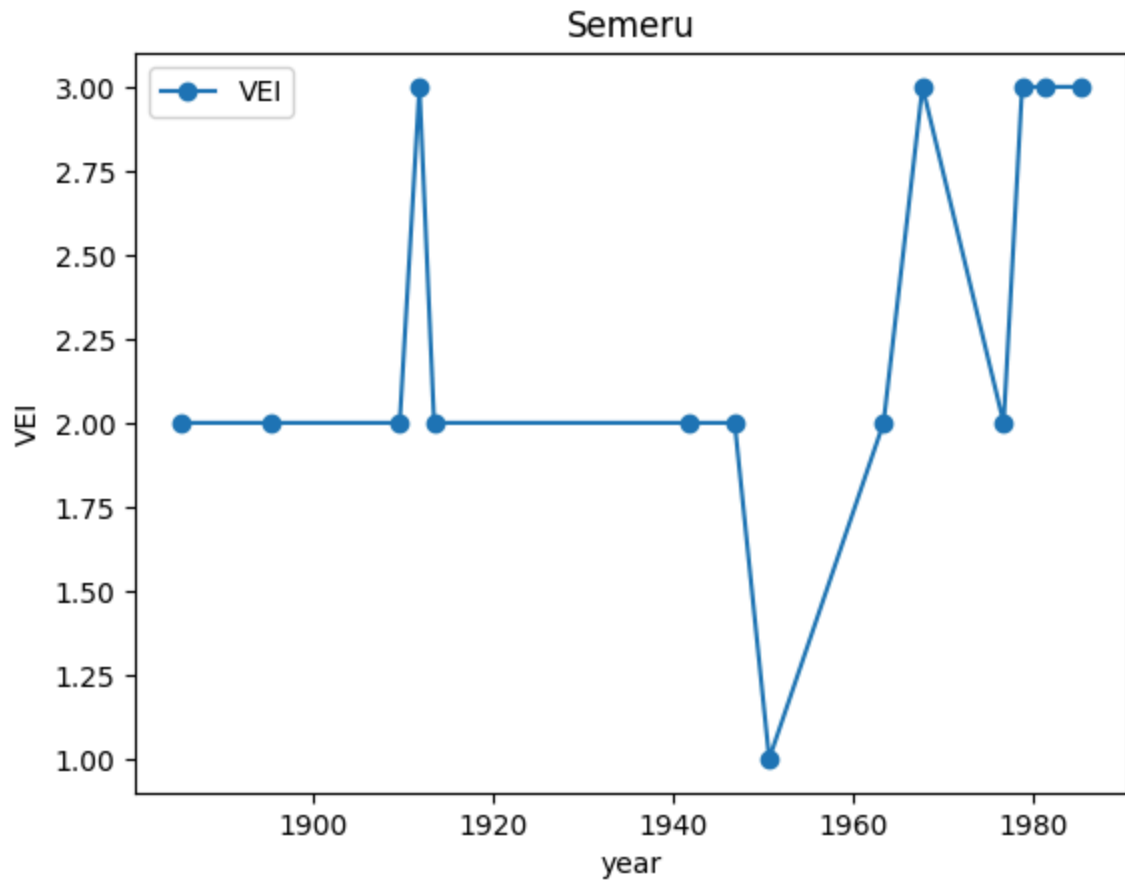
```
mpl.title(volcano)
mpl.xlabel('year')
mpl.ylabel('VEI')
mpl.legend(['VEI'])
mpl.show()
```

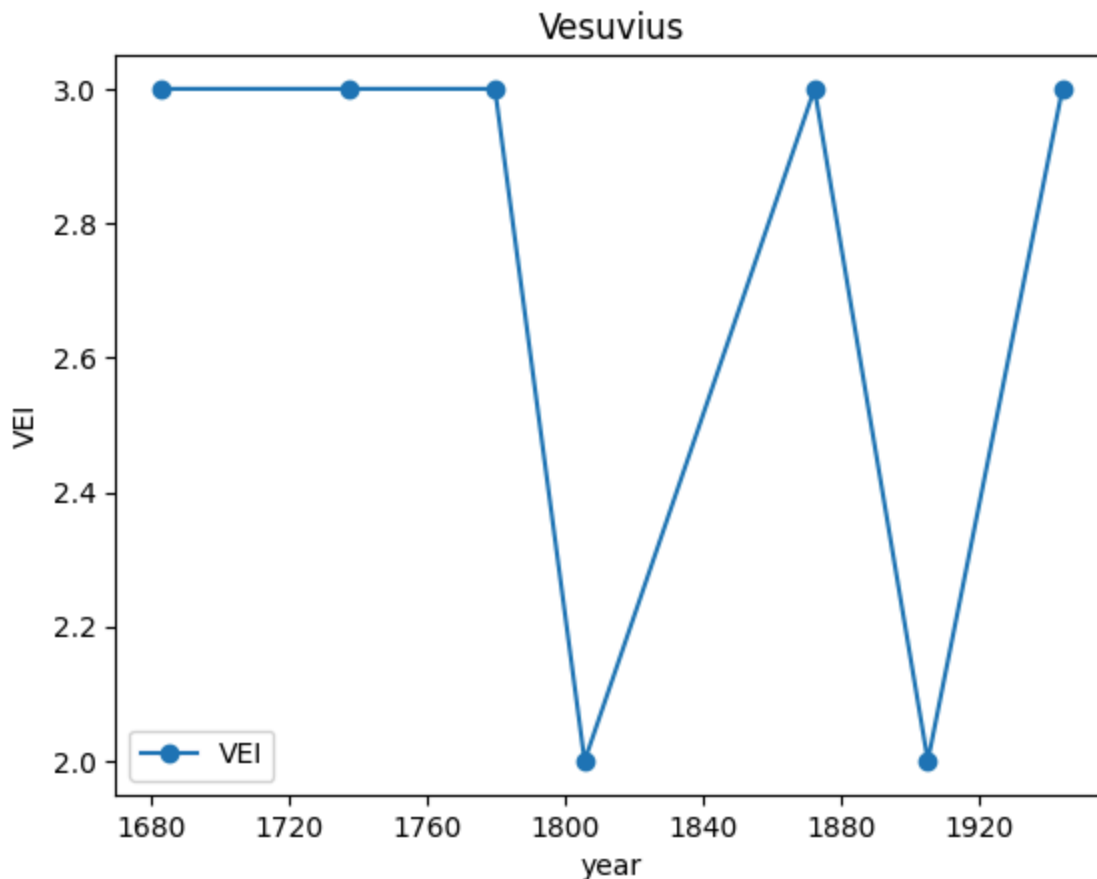












Part 3: City Jobs (15 POINTS TOTAL)

Proficiency in SQL is also super important. SQL databases are essentially relational databases in which there are vast amounts of tabular data, which can often be used to connect with related tabular data. [This](#) is a pretty good intro into learning more about SQL.

Check out this [tutorial](#) for some clarifications on SQL.



IMPORTANT: Downloading the Dataset

Please follow these steps to download the database file:

- **Step 1:** Go to the Kaggle Dataset
 - Click this link: [SF Salaries Dataset on Kaggle](#)

- **Step 2:** Create/Login to Kaggle Account
 - You need a free Kaggle account to download datasets
 - If you don't have one, sign up at kaggle.com
 - Log in to your account
- **Step 3:** Download and Extract
 - On the dataset page, click the "Download" button (upper right corner)
 - A ZIP file will be downloaded. Locate it in your system and extract it.
 - You will see a `database.sqlite` file - this is what we need.
 - Move the `database.sqlite` file to the same folder where your notebook is saved.

Now! We'll be using `sqlite` to access a database.

- Check out the description of the data so you know the table / column names.

The following code will use `sqlite3` to create a database connection. `sqlite3` is the library in Python that assists in navigating through SQL databases.

Note: If you are working on this assignment via Google Colab, sometimes the runtime resets and it will throw errors.

Instead of running through the entire notebook, run the notebook from the following code block and onwards:

- Click anywhere on the next code block.
- Go up to where it says **'Runtime'** in the toolbar (right under the title of the notebook and **in between 'Insert' and 'Tools'**)
- Hover over it and **click on the option** that says **'Run cell and below'**

```
In [3]: import sqlite3
import pandas as pd # Pandas was already imported from the previous sections

conn = sqlite3.connect("database.sqlite")
crsr = conn.cursor()
```

If you cannot see anything when executing this code, that means you did not download the Kaggle dataset correctly.

```
In [4]: # This code will let you check out the different tables within the database.
query = "SELECT name FROM sqlite_master WHERE type='table';"
tables = crsr.execute(query).fetchall()
print(tables)
```

```
[('Salaries',)]
```

Remember that each problem should be solved with a single SQL query.

Note: All outputs must be shown

- Only include whatever fields are mentioned throughout each question, nothing more and nothing less.
- Follow each instruction clearly

3.1.1: 2 Points

From the **Salaries** table, get the **average total pay** for all employees whose job titles contain the word **"clerk"** (**not case-sensitive**) between the years **2011 to 2014**.

Remember that any employee with "clerk" in their job title should be included, regardless of other words in the title.

Hint: Look into [this](#)

```
In [5]: query = '''
SELECT AVG(TotalPay)
FROM Salaries
WHERE JobTitle LIKE '%clerk%'
AND Year BETWEEN 2011 AND 2014
'''

# KEEP THIS. It will display the whole dataframe.
df = pd.read_sql(query, conn)
df
```

```
Out [5]:  AVG(TotalPay)
0  48086.855056
```

3.1.2: 2 Points

From the **Salaries** table, get all the clerks (all job titles consisting of the word "clerk" (**not case-sensitive**)) in the **year 2013** making under **\$50,000 as a Total pay**. **Sort** them in **descending** order by their pay.

Remember that clerks that also occupy other professions are still considered clerks.

```
In [7]: query = '''
SELECT *
FROM Salaries
WHERE JobTitle LIKE '%clerk%'
AND Year = 2013
AND TotalPay < 50000
ORDER BY TotalPay DESC
'''

# KEEP THIS. It will display the whole dataframe.
df = pd.read_sql(query, conn)
df
```

Out [7]:

	Id	EmployeeName	JobTitle	BasePay	OvertimePay	OtherPay	Benefit
0	98430	Linda A Sabau	Senior Clerk	47570.1	275.31	2103.83	25454.9
1	98651	Laurie A Kassel	Clerk	49316.45	0.00	606.66	24224.1
2	98437	Roberto O Ayala-Duran	Assessment Clerk	48735.83	101.76	1080.00	25450.5
3	98994	Norma C Morales	Clerk Typist	48206.65	665.44	985.00	21798.6
4	98971	Candice Y Ye	Senior Account Clerk	49831.41	0.00	0.00	21964.0
...	...	...	...	...	...	...	.
782	106932	Ana D Espinoza	Clerk Typist		0.00	0.00	7959.1
783	106935	Priscilla Kartawidjaya	Clerk Typist		0.00	0.00	7959.1
784	106944	Ben P Gallo	Chf Payroll & Personnel Clerk		0.00	0.00	7959.1
785	106974	Yao Zhang Mai	Senior Clerk Typist		0.00	0.00	7959.1
786	106978	Cathie L Serletic	Clerk Typist		0.00	0.00	7959.1

787 rows x 13 columns



3.1.3: 4 Points

From the **Salaries** table, first get the **averages** of **base pay**, **benefits**, and **overtime pay** for clerks (all job titles consisting of the word "clerk" **(not case-sensitive)**).

- Then, make a **column with the sum** of these **three averages**
- Finally, **exclude** job titles containing "SENIOR" **(case-sensitive)**

Remember that clerks that also occupy other professions are still considered clerks.

Example:

Let's say the Salaries table has these records:

JobTitle	BasePay	Benefits	OvertimePay
CLERK	40000	15000	5000

JobTitle	BasePay	Benefits	OvertimePay
Clerk	35000	12000	3000
ACCOUNTING CLERK	45000	18000	6000
SENIOR CLERK	60000	25000	8000
Senior Clerk	55000	22000	7000
MANAGER	80000	30000	10000

Which rows to include:

- Rows 1, 2, 3, 5 (all contain "clerk" in any case)
- Exclude Row 4 (contains uppercase "SENIOR")
- Exclude Row 6 (doesn't contain "clerk")

Calculate averages from rows 1, 2, 3, 5:

- $\text{avg_base_pay} = (40000 + 35000 + 45000 + 55000) / 4 = 43750$
- $\text{avg_benefits} = (15000 + 12000 + 18000 + 22000) / 4 = 16750$
- $\text{avg_overtime_pay} = (5000 + 3000 + 6000 + 7000) / 4 = 5250$
- $\text{total_avg_compensation} = 43750 + 16750 + 5250 = 65750$

Your output:

avg_base_pay	avg_benefits	avg_overtime_pay	total_avg_compensation
43750	16750	5250	65750

Note: Actual values from the database will be different!

```
In [8]: query = '''
SELECT AVG(BasePay) AS avg_base_pay,
AVG(Benefits) AS avg_benefits,
AVG(OvertimePay) AS avg_overtime_pay,
AVG(BasePay) + AVG(Benefits) + AVG(OvertimePay) AS sum_of_averages
FROM Salaries
WHERE JobTitle LIKE '%clerk%'
AND JobTitle NOT LIKE '%SENIOR%'
'''

# KEEP THIS. It will display the whole dataframe.
df = pd.read_sql(query, conn)
df
```

```
Out[8]:
```

	avg_base_pay	avg_benefits	avg_overtime_pay	sum_of_averages
0	45188.292242	15170.247828	1075.427168	61433.967237

3.1.4: Creating Year-Based Tables (7 Points)

Finally, we'll make our own year based- table in our database. We already have the "**Salaries**" table above that contains data for multiple years (2010–2015).

Task: Create separate tables for each year (2010–2015) and add them back to the database.

- Split the Salaries table into **six year-based tables (2010–2015)**.
- Each table should contain **only the rows from that year**.
- You **must use SQL queries** to retrieve data from the Salaries table.
- You may use Python and pandas to store results and **write tables back** to the database.
- Using a Python **loop** may be helpful.
- You may use multiple lines of python code for this problem.

Hint: Check [this](#) out

Think about how you've already used SQL to read data into pandas, and how pandas can write DataFrames back into a database.

```
In [10]: for year in range(2010, 2016):
          query = f"SELECT * FROM Salaries WHERE Year = {year}"
          year_df = pd.read_sql(query, conn)
          year_df.to_sql(f"Salaries_{year}", conn, if_exists='replace', index=False)
```

```
In [11]: # Run this code to check if you successfully added your table.
          cursor = conn.cursor()
          cursor.execute("SELECT name FROM sqlite_master WHERE type='table';")
          print(cursor.fetchall())
```

```
[('Salaries',), ('Salaries_2010',), ('Salaries_2011',), ('Salaries_2012',),
 ('Salaries_2013',), ('Salaries_2014',), ('Salaries_2015',)]
```



Part 4: Pandas 'Group By' (50 points)

This flowchart is taken from our lecture class presentation and illustrates the process of transforming data using the Pandas GroupBy operation. First, the data is input, followed by applying the GroupBy function to one or more columns of the DataFrame. Once the data is grouped, an aggregation function (such as sum(), mean(), or count()) is applied to compute summary statistics for each group



Your task is to translate the workflow shown in the flowchart into Pandas queries that perform these operations step by step.

Notes: Your task is to translate the workflow shown in the flowchart into Pandas queries. Ensure that the **exact input and exact output** from the flowchart are replicated using Pandas queries, step by step.

4.1 (Point 10) Create a sample dataset that includes columns for account, order, and ext price.

```
In [13]: # Create the Sample dataset from above flowchart
input_data = {
    'account': ['383080', '383080', '383080', '412290', '412290', '412290', '412290', '412290'],
    'order': ['10001', '10001', '10001', '10005', '10005', '10005', '10005', '10005'],
    'ext_price': [235.83, 232.32, 107.97, 2679.36, 286.02, 832.95, 3472.04, 3472.04]
}

# Create DataFrame
df = pd.DataFrame(input_data)

# Display Dataframe (DONT REMOVE THE CODE)
pd
```

```
Out[13]: <module 'pandas' from '/Users/riyatakalkar/Library/Python/3.9/lib/python/site-packages/pandas/__init__.py'>
```

4.2 (Point 10+10 =20) Group by order and show the intermediate results

```
In [15]: # Group by 'order' and show intermediate result
grouped = df.groupby('order')

# Display intermediate result for each group; hints: you have to use 'for loop'
for order, group in grouped:
    display(group)
```

	account	order	ext_price
0	383080	10001	235.83
1	383080	10001	232.32
2	383080	10001	107.97

	account	order	ext_price
3	412290	10005	2679.36
4	412290	10005	286.02
5	412290	10005	832.95
6	412290	10005	3472.04
7	412290	10005	915.12

	account	order	ext_price
8	218895	10006	3061.12
9	218895	10006	518.65
10	218895	10006	216.90
11	218895	10006	-72.18

4.3 (Point 10) Apply the Sum Aggregation for Each Group

Now we'll apply the sum aggregation to get the total ext price for each order:

```
In [16]: # Repeat group by 'order' again and then apply aggregation (sum of 'ext_price')
aggregated_sum = df.groupby('order')['ext_price'].sum()

# Show the aggregated result after re-grouping (DONT REMOVE THE CODE)
aggregated_sum
```

```
Out[16]: order
10001      576.12
10005     8185.49
10006     3724.49
Name: ext_price, dtype: float64
```

4.4 (Point 10) Combine the Results into One Final Table Finally, we will **reset the index** and create a combined table that shows order and the sum of the ext price for each group:

Notes: In pandas, 'reset_index()' is a method used to reset the index of a DataFrame to its default integer-based index. By default, when you perform certain operations like groupby(), the resulting DataFrame may have a new index (e.g., the grouped column). The reset_index() method allows you to convert the current index back to a default sequential integer index and optionally, move the current index values into a regular column.

```
In [17]: # Reset index to combine result into a single DataFrame
final = aggregated_sum.reset_index()
# Rename the columns for clarity
final.columns = ['order', 'ext_price_sum']
```

```
# Show the final result (DONT REMOVE THE CODE)
final
```

```
Out[17]:
```

	order	ext_price_sum
0	10001	576.12
1	10005	8185.49
2	10006	3724.49

Bonus(1 point): Assignment Submission with Jupyter Notebook file(ipynb) and PDF

In addition to the instruction of the special bonus question included exclusively in Assignment 1, we would like to highlight the importance of developing good practices for assignment submission in data science projects.

Think of yourself as a reviewer of your peers' work: what would you pay attention to if you had to grade their assignments? Clearly, the raw dataset itself is not the main point—why would someone need to see every line of data you downloaded? Data science is about analysis and insight, not just data collection. What truly matters is how you analyze the data and how you present your results.

For a clean and effective submission, please avoid printing entire datasets unless they are directly relevant to the presentation of your output or if you are asked to do so. Instead, focus on:

- Showing your algorithm with clear, concise code
- Displaying the key results you want us to evaluate (or the outputs you relied on while working)
- Adding a short explanation when asked to interpret your findings

For this bonus question, please submit both your .ipynb file and the PDF version. To create a PDF: go to File (top left corner in Jupyter Notebook) → Print → Destination → Save as PDF.

Are you noticing an overwhelming amount of unnecessary printed results in your PDF? This is exactly why we emphasize the importance of cleaning up your work before submission. Removing trivial data printing makes your notebook look much more professional and easier to review—just like proofreading an essay for good grammar.

Your Homework 1 submission should be lightweight and focused, since this is a starter project designed to help you ease into the semester.

This practice helps us grade more efficiently and ensures fairness, since we can always reference your original notebook if regrading is needed.